INTERN ASSIGNMENT

AI Simulation — Technical Task

Economic agents · Social dynamics · LLM policymaker

Time allowed	3–4 days
Submit	GitHub repo link + 1-page PDF writeup
Language	Python 3.10+

READING

Two papers to skim before you start

You don't need to read every equation. Focus on understanding the core setup: what a simulation environment looks like, how agents behave, and what a policymaker (principal) is trying to do.

Paper 1 — Social Environment Design (Zhang et al., ICML 2024)
Read: Abstract, Section 1 (Introduction), Section 4 (the Apple Picking example). Skim the rest.

https://arxiv.org/abs/2402.14090

Paper 2 — Large Legislative Models (Gasztowtt et al., 2024)
Read: Abstract, Section 1, Section 4 (how the LLM principal works — Figure 3 is the key diagram), Section 5.1 (the three environments).

https://arxiv.org/abs/2410.08345

> **The core idea in plain English**
> You have a group of self-interested agents sharing a common resource. Left alone, they make individually rational choices that collectively make things worse for everyone — a classic tragedy of the commons. A policymaker sits above the simulation and tweaks the rules each round (e.g. tax rates, incentives) to steer agents toward a better collective outcome. Your job is to build this, then use an LLM as the policymaker.

THE TASK

Build a commons harvest simulation with an LLM policymaker

You will build a simplified version of the commons harvest environment from the papers, then put two different policymakers on top of it and compare them.

The environment

A shared apple field. 10 agents. Each round, every agent decides how aggressively to harvest — a number between 0 (gentle) and 1 (maximum harvest). The field has a health score that goes up when agents harvest gently and goes down when they overharvest. If the field health drops below 30%, all rewards are halved until it recovers.

Agent behaviour

Each agent has a fixed greed level (a number between 0 and 1, assigned randomly at the start). A greedy agent defaults to high harvest. A less greedy agent is more willing to hold back — but only if the incentives

make it worthwhile. Agents update their harvest rate each round based on the tax they face and their own greed level. Keep the update rule simple: the agent softmax-chooses between low and high harvest based on expected after-tax reward.

The policymaker

The policymaker sets one thing each round: a tax rate on harvesting (a number between 0 and 1). Agents who harvest a lot pay more tax; the collected tax is redistributed equally to all agents. The policymaker sees the current field health, the average harvest rate last round, and the average agent reward last round. It does not see individual agent greed levels.

> **One objective to optimise**
> Total welfare = field health. That's it. The policymaker's job is to keep field health as high as possible over 150 rounds. Agent rewards are a side effect, not the target. A policymaker that maximises field health at the cost of zero agent reward has failed — agents will stop participating. Build this tension in.

What to build

- **The simulation** — a clean Python class. Seeded and reproducible. The spec above has gaps; fill them with reasonable choices and document what you assumed.
- **A rule-based policymaker** — a simple hand-coded policy you design yourself. For example: raise tax when field health drops below a threshold, lower it when health is high. Explain your logic in a comment.
- **An LLM policymaker** — at the start of each round, build a prompt with three parts: (1) a description of the situation, (2) the history of the last 5 rounds (tax rate, field health, average reward), (3) a question asking what tax rate to set next. Feed this to any LLM API. Use the number it returns as the tax rate. This is the method from Paper 2, Section 4.
- **Comparison** — run both policymakers for 150 rounds, 5 seeds each. Plot field health over time for both. Report average field health in the final 50 rounds. Log the total API cost for the LLM runs.

```
# The interface your simulation should expose
sim = CommonsSim(n_agents=10, seed=42)
obs = sim.reset() # {'field_health': 1.0, 'avg_harvest': 0.0, 'avg_reward': 0.0}
for round in range(150):
tax_rate = policymaker.act(obs) # number between 0 and 1
obs, field_health, info = sim.step(tax_rate)
```

SCORING

100 points

Component	Pts	What we look for
Simulation correctness	30	Field dynamics work, degraded state triggers correctly, seeds reproduce.
Rule-based policymaker	15	Has a clear logic. Better than random. Explained in comments.
LLM policymaker	25	Prompt has the 3-part structure from Paper 2. Cost is logged.

Comparison & plots	20	Clean plot, correct metric, honest about which performed better.
Writeup (1 page hard limit)	10	One finding, one thing you'd change, one question the task raised.

A FEW NOTES

Read this before you start

• The spec leaves some things open on purpose — for example, the exact formula for how agents choose their harvest rate. Make a decision, document it, move on. We are not looking for the 'right' answer. We are looking at how you handle ambiguity.

• Your rule-based policymaker does not need to beat the LLM. It needs to be *reasoned*. A dumb policy with a clear explanation beats a clever one with none.

• If the LLM performs no better than a fixed tax rate of 0.5, say so and explain why. Honest null results are fine.

• We will read your commit history. Commit as you go.

Questions? Email with a specific subject line describing exactly what you're stuck on.